MONITOR Z · L01.5

Clase 5 — Hablar con el sistema y leer el listado: macros, SVC, DCB, AMODE y RMODE

Cuaderno de estudio · versión del 2026-10-28

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L01.5 — Clase 5 · Hablar con el sistema y leer el listado: macros, SVC, DCB, AMODE y RMODE

Temario de la L01: 1. La forma de un programa · 2. Los registros y las instrucciones de todos los días · 3. Entrar y salir: prólogo, save area, epílogo y RC · 4. Los datos: DS, DC, literales, el PARM y el `EX` · **5. Hablar con el sistema y leer el listado: macros, SVC, DCB, AMODE/RMODE**.

Objetivo de esta clase: entender cómo un programa le pide cosas a z/OS (abrir, escribir, avisar, cerrar) y leer un listado **completo**, de la página 1 a la 11, más el mapa del binder. Al terminar tenés que poder:

- decir qué hace por dentro cada una de tus 5 macros y cuáles terminan en un `SVC`;
- ubicar los campos principales de un DCB y explicar qué cambia el `OPEN`;
- explicar `AMODE 31` / `RMODE 24` y por qué ZTKHOLA y ZTKMON los necesitan;
- encontrar en el listado lo que buscás: símbolos, registros, direcciones que hay que reubicar, errores.

> 📚 Fuentes: - `monitor_z/asm/ZTKHOLA.asm`, renglones 1–3, 27–30, 61, 69 y 94–95. - El listado de **JOB24962**: sentencias 27–44, 75–83, 91–97 y 122–163, y las páginas 1–11. - `LKED/SYSPRINT` de JOB24962: el *Module Map* y los *Save Module Attributes*. - El listado de **JOB34775** (ZTKMON v0.9): *Macro and Copy Code Source Summary* (página 224) y sentencias 10158–12237. - *z/OS DFSMS Macro Instructions for Data Sets* (OPEN, PUT, CLOSE, DCB); *z/OS MVS Assembler Services Reference* (WTO, STORAGE).

Respuestas de la clase 4

1. `CONTADOR DS H` caería en `0D4`–`0D5`. `SAVEAREA DS 18F` necesita un múltiplo de 4, así que el ensamblador salta a `0D8` y deja un **hueco de 2 bytes**. Todo lo que sigue se corre 4 bytes.
2. Sin `LTORG`, el ensamblador pone el pool de literales **al final** del programa. Como el `LTORG` ya estaba en el último renglón antes del `END`, quedarían en el mismo lugar y **no cambiaría ningún** `Loc`. `LTORG` importa cuando

hay código o datos **después**: por ejemplo, para que los literales queden al alcance de la base.

3. Con `PARM='A'`, R3 = 1 → `BCTR` → 0 → `D200`: copia 1 byte. Con 150 caracteres, `CH` deja CC 2 («mayor»), `BNH` **no** salta y `LH 3,=H'100'` pone el tope: R3 = 100 → `BCTR` → 99 = `X'63'` → `D263`, que copia 100 bytes. (El JCL no deja pasar más de 100 caracteres de PARM, pero el programa no confía en eso.)

1. Una macro es un pedido armado por IBM

Tu programa no sabe abrir un archivo. **z/OS sí.** La macro arma el pedido con el formato que z/OS espera, para que no lo tengas que escribir a mano. Ya viste que se expande con `+` (clase 1). Ahora fijate en la **columna de la derecha** del listado:

```
000020 4D10 C028      00028    30+         BAS   1,*+8
00009A 4110 C22C      0022C    79+         LA    1,PRTDCB
```

`01-OPEN` quiere decir «nivel 1, lo trajo la macro `OPEN`». `02-IHBI` es nivel **2**: una macro (`IHBINNRA`) que la macro `PUT` llamó por dentro. La página 8 del listado (*Macro and Copy Code Source Summary*) lista las **8** que se usaron, todas de `SYS1.MACLIB`: `CLOSE`, `DCB`, `IHBINNRA`, `IHB01`, `OPEN`, `PUT`, `SYSSTATE` y `WTO`.

ZTKMON usa muchas más: la página 224 de su listado lista **25** de `SYS1.MACLIB` y **5** de `SYS1.MODGEN`. Algunas son pedidos, como las tuyas (`STORAGE`, `STIMER`, `TIME`), y otras son **mapas** de los bloques de control de z/OS que el monitor lee (`CVT`, `IHAASCB`, `IHAASVT`…): no generan código, describen campos (L15).

2. El SVC: la ventanilla de z/OS

Un programa común **no puede** abrir un archivo por su cuenta: eso lo hace el sistema, con permisos que tu programa no tiene. Para pedírselo, la CPU tiene una instrucción especial: **`SVC n`** (*supervisor call*). Es como tocar el timbre de una **ventanilla** con un número: la CPU deja tu programa, entra al supervisor, z/OS atiende el pedido número *n* y te devuelve el control en la instrucción siguiente.

Tu macro	Cómo termina	Bytes	Qué le pasa a z/OS
`OPEN (PRTDCB,OUTPUT)`	`SVC 19`	`0A13`	en R1, una lista: `8F` (OUTPUT, último) + `AL3(PRTDCB)`
`WTO 'ZTKHOLA: …'`	`SVC 35`	`0A23`	en R1, el mensaje: largo 38, flags, 34 caracteres
`CLOSE (PRTDCB)`	`SVC 20`	`0A14`	en R1, una lista: `80` (último) + `AL3(PRTDCB)`
`PUT PRTDCB,SALIDA`	**no** es un SVC: `BASR 14,15`	`0DEF`	salta a una rutina de QSAM que **el `OPEN` dejó preparada**
`DCB …`	**no ejecuta nada**	—	son 96 bytes de **datos** que describen el archivo

`0A` es el código de la instrucción `SVC`, y el segundo byte es el **número** de la ventanilla: `X'13'` = 19, `X'23'` = 35, `X'14'` = 20.

Por qué `PUT` no usa SVC: escribir es lo que más se repite (una vez por línea). Pasar por el supervisor cada vez sería caro. Por eso el `OPEN` hace el trabajo pesado una sola vez y deja **en el DCB** la dirección de una rutina que escribe en un buffer. El `PUT` sólo la llama:

```
0000A2 1FFF                    81+         SLR   15,15               CLE
0000A4 BFF7 1031      00031    82+         ICM   15,7,49(1)          LOA
0000A8 0DEF                    83+         BASR  14,15               LIN
```

`ICM 15,7,49(1)` trae a R15 los **3 bytes** que hay en DCB+49 (R1 apunta al DCB): la dirección de la rutina `PUT`. `BASR 14,15` salta ahí y deja en R14 por dónde volver: la misma convención de la clase 3.

En ZTKMON aparece un tercer camino: `STORAGE OBTAIN` (sentencia 12181) no usa `SVC`, sino `PC` (*program call*, `B218`, sentencia 12195), otra puerta al sistema. Y `STIMER` (la espera entre ciclos) es `SVC 47` (`0A2F`, sentencia 11558).

El SVC: la ventanilla de z/OS

L01 · Clase 5 · Hablar con el sistema

Tu macro	Termina en	Bytes	Qué le deja a z/OS en R1
`OPEN (PRTDCB,OUTPUT)`	`SVC 19`	`0A13`	lista: `8F` (OUTPUT, último) + `AL3(PRTDCB)`
`WTO 'ZTKHOLA: …'`	`SVC 35`	`0A23`	el mensaje: largo 38, flags, 34 caracteres
`CLOSE (PRTDCB)`	`SVC 20`	`0A14`	lista: `80` (último) + `AL3(PRTDCB)`
`PUT PRTDCB,SALIDA`	**sin SVC**: `BASR 14,15`	`0DEF`	R1 = el DCB, R0 = la línea
`DCB …`	no ejecuta nada	—	96 bytes de datos

🔔 `0A` + número

El segundo byte es el número de ventanilla: `X'13'` = 19, `X'23'` = 35, `X'14'` = 20.

🏃 Por qué PUT no

Escribir se repite en cada línea. `OPEN` deja en DCB+49 la rutina, y `PUT` la llama: `ICM 15,7,49(1)` + `BASR 14,15`.

🕊️ En ZTKMON

`STIMER` = `SVC 47` (`0A2F`). `STORAGE OBTAIN` usa otra puerta: `PC` (`B218`).

> Columna de la derecha del listado: `01-OPEN` = nivel 1, lo trajo `OPEN` · `02-IHBI` = nivel 2, una macro que `PUT` llamó por dentro. Página 8: 8 macros de `SYS1.MACLIB`.

Fuente: listado de JOB24962 (sentencias 29–44, 79–83, 93–97; página 8) · listado de JOB34775 (sentencias 11558, 12181–12195)

Lámina L01-26_la_ventanilla

3. El DCB por dentro: 96 bytes que describen SYSPRINT

`PRTDCB DCB DDNAME=SYSPRINT,DSORG=PS,MACRF=PM,RECFM=FB,LRECL=133,BLKSIZE=13300` se volvió las sentencias 125–163: un bloque de **96 bytes** (`22C`–`28B`). Cada parámetro tuyo cae en un campo:

Desplazamiento	Loc	Campo	Bytes en el listado	Qué quiere decir	Lo pediste con
+26	`246`	DSORG	`4000`	secuencial (PS)	`DSORG=PS`
+36	`250`	RECFM	`90` = `1001 0000`	**F**ijo + **B**loqueado	`RECFM=FB`
+40	`254`	DDNAME	`E2E8E2D7D9C9D5E3`	`SYSPRINT` en EBCDIC	`DDNAME=SYSPRINT`
+48	`25C`	OFLGS	`02`	las banderas del `OPEN`	—

+50	`25E`	MACR	`0050`	se va a usar `PUT` en modo *move*	`MACRF=PM`
+62	`26A`	BLKSIZE	`33F4` = 13.300	tamaño del bloque	`BLKSIZE=13300`
+82	`27E`	LRECL	`0085` = 133	largo del registro	`LRECL=133`

Lo que cambia el `OPEN`: el DCB que ves en el listado es **cómo está antes de abrir**. `OPEN` lo completa:

- prende el bit `X'10'` de **OFLGS** (`DCBOFOPN`) si abrió bien. Es lo que pregunta tu `TM PRTDCB+48,X'10'` del renglón 28 (`+48` = `X'30'`, y `22C + 30 = 25C`);
- escribe en **+49** la dirección de 3 bytes de la rutina `PUT`, encima de lo que había. Es la que lee el `ICM 15,7,49(1)`.

> 💡 El JCL de RUN1 también trae `DCB=(RECFM=FB,LRECL=133,BLKSIZE=13300)` en el DD SYSPRINT. Cuando el programa ya completó un campo del DCB, **manda el programa**; el JCL sólo llena los que el programa dejó vacíos. Acá dicen lo mismo, así que no hay conflicto que medir.

El DCB por dentro: 96 bytes que describen SYSPRINT

L01 · Clase 5 · Hablar con el sistema

+	Loc	Campo	Bytes	Qué es	Lo pediste con
26	`246`	DSORG	`4000`	secuencial	`DSORG=PS`
36	`250`	RECFM	`90` = `1001 0000`	Fijo + Bloqueado	`RECFM=FB`
40	`254`	DDNAME	`E2E8E2D7D9C9D5E3`	«SYSPRINT»	`DDNAME=SYSPRINT`
48	`25C`	OFLGS	`02`	banderas del OPEN	—
50	`25E`	MACR	`0050`	PUT en modo move	`MACRF=PM`
62	`26A`	BLKSIZE	`33F4` = 13.300	tamaño del bloque	`BLKSIZE=13300`
82	`27E`	LRECL	`0085` = 133	largo del registro	`LRECL=133`

🔓 Lo que cambia el OPEN

Prende `X'10'` en OFLGS (+48 = `25C`): es lo que pregunta tu `TM PRTDCB+48,X'10'`. Y escribe en +49 la dirección de la rutina `PUT`.

⚖️ Programa vs. JCL

El DD SYSPRINT también trae `DCB=(RECFM=FB,LRECL=133,BLKSIZE=13300)`. Lo que el programa ya completó, manda; el JCL llena lo vacío. Acá coinciden.

Fuente: listado de JOB24962 (sentencias 125–163, 28, 82) · JESJCL de JOB24962 · DFSMS Macro Instructions for Data Sets

Lámina L01-27_el_dcb

4. AMODE y RMODE: con cuántos bits se mira y dónde se vive

Los renglones 2 y 3 dicen `ZTKHOLA AMODE 31` y `ZTKHOLA RMODE 24`. Son dos cosas distintas:

	Pregunta que contesta	ZTKHOLA	Qué significa
AMODE (*addressing mode*)	¿con cuántos bits **calcula direcciones** mientras corre?	**31**	puede alcanzar hasta 2 GB
RMODE (*residency mode*)	¿**dónde se puede cargar** el programa?	**24**	debajo de la «línea» de 16 MB

Analogía: AMODE es **hasta dónde ves** desde tu oficina; RMODE es **en qué piso te dejan tener la oficina**. ZTKHOLA ve todo el edificio (31 bits), pero vive en la planta baja (debajo de 16 MB).

Por qué RMODE 24: el DCB vive **adentro** del programa, y QSAM lo exige **debajo de la línea**, porque varios campos del DCB son direcciones de **3 bytes** (24 bits). Lo ves en el propio listado: `AL3(PRTDCB)` en las listas del `OPEN` y del `CLOSE`, y el `ICM 15,7,…` que trae 3 bytes. Con 3 bytes no se puede escribir una dirección de más de 16 MB. Es lo que dicen tus comentarios de los renglones 80–81.

El binder lo confirma (LKED/SYSPRINT de JOB24962): `AMODE 31`, `RMODE 24`, `EXCEEDS 16MB NO`, y en el *Entry Point Summary*, `ZTKHOLA MAIN_EP 31 00000000`.

ZTKMON hace lo mismo (sentencias 2–3: `AMODE 31`, `RMODE 24`) por la misma razón (su comentario de la sentencia 12829: *«El DCB va en la CSECT, que es RMODE 24: QSAM lo exige bajo la línea»*). Pero lo grande, como los buffers, lo pide con `STORAGE OBTAIN,LOC=31`: **arriba** de la línea, donde hay lugar.

AMODE y RMODE: hasta dónde ves y en qué piso vivís

L01 · Clase 5 · Hablar con el sistema

AMODE 31

con cuántos bits **calcula direcciones** al correr: hasta 2 GB

RMODE 24

dónde se puede cargar: debajo de la línea de 16 MB

El binder lo confirma

`AMODE 31` · `RMODE 24` · `EXCEEDS 16MB NO` · `MAIN_EP 31`

	ZTKHOLA	ZTKMON v0.9
Renglones	2–3: `AMODE 31` , `RMODE 24`	sentencias 2–3: lo mismo
Por qué RMODE 24	el DCB va **adentro** y QSAM lo exige debajo de la línea: sus direcciones son de 3 bytes (`AL3(PRTDCB)` , `ICM 15,7`)	su comentario (sent. 12829): «QSAM lo exige bajo la línea»
Lo grande	—	`STORAGE OBTAIN,LOC=31` : arriba de la línea

> Con 3 bytes no se puede escribir una dirección de más de 16 MB (`X'FFFFFF'`). Por eso el programa que tiene el DCB vive abajo, aunque mire con 31 bits.

Fuente: ZTKHOLA.asm (renglones 2–3, 80–81) · LKED/SYSPRINT de JOB24962 · listado de JOB34775 (sentencias 2–3, 12181, 12829)

Lámina L01-28_amode_rmode

5. El listado completo, página por página

Página	Sección	Para qué la mirás	En JOB24962
1	*Option Summary*	con qué opciones se ensambló	`XREF(SHORT,UNREFS)` ; el JCL pidió `OBJECT,NODECK`
2	*External Symbol Dictionary* (ESD)	qué secciones y nombres externos tiene el objeto	`ZTKHOLA SD … 00000000 000002A9` : una sola sección, de 681 bytes
3–5	el fuente con los bytes	todo lo de las clases 1 a 4	170 sentencias
6	*Relocation Dictionary* (RLD)	qué direcciones **absolutas** hay que corregir al cargar	**2**: `000025` y `0000B5` , de 3 bytes
7	*Ordinary Symbol and Literal Cross Reference*	dónde se define y dónde se usa cada nombre	`RCFINAL` … `20M 45M 84M 101`
8	*Macro and Copy Code Source Summary*	de dónde salió cada macro	8 de `SYS1.MACLIB`
9	*Using Map*	hasta dónde llegó cada base	R12 hasta el desplazamiento `29E`

10	*General Purpose Register Cross Reference*	quién usa y quién cambia cada registro	R4–R11 sólo en 12 y 105 (clase 3)
11	*Diagnostic Cross Reference and Assembler Summary*	los errores y el resumen	`No Statements Flagged` · `Return Code 000`

La RLD, el contraste con la clase 2. Casi todas tus direcciones son **base + desplazamiento**, y no necesitan corrección: cambia R12 y listo. Pero hay **dos** que son direcciones **escritas en los datos**: `AL3(PRTDCB)` en la lista del `OPEN` (`Loc 000025`) y en la del `CLOSE` (`0000B5`). El ensamblador escribió `00022C`, que es la distancia desde el principio. Al cargar el programa, z/OS le **suma** la dirección donde lo cargó (por eso la `Action` es `+`). Por eso la RLD tiene exactamente dos renglones.

Las letras de la Cross Reference: `M` = la sentencia **modifica** el dato (`20M`: el `MVC` escribe en `RCFINAL`), `B` = es destino de un **salto** (`ABRIO … 35B`), `U` = un `USING` (`ZTKHOLA … 14U`), `X` = destino de un `EX` (`COPIAPRM … 74X`). Un número sin letra es una lectura (`RCFINAL … 101`: el `L 15,RCFINAL`).

Y después, el binder (LKED/SYSPRINT):

Qué dice	Qué es
`CLASS B_TEXT LENGTH = 2A9 … RMODE= 24`	tus 681 bytes de código y datos
`ALIGN = DBLWORD`	el bloque se alinea a **doble palabra** (8 bytes)
`MODULE SIZE (HEX) 000002B0`	688 bytes
`IEW2008I … RETURN CODE = 0`	el `LKED 00` del JESMSGLG

> 🔎 **La pista del relleno de la clase 1.** Allá quedó sin medir por qué 681 bytes se volvieron 688. El mapa del binder dice `ALIGN = DBLWORD`, y **688 es el primer múltiplo de 8 desde 681**. Es la explicación más probable, pero sigue siendo una inferencia: ningún mensaje del binder dice «agregué 7 bytes de relleno».

El listado completo, página por página

L01 · Clase 5 · Hablar con el sistema

Pág.	Sección	Para qué	En JOB24962
1	Option Summary	con qué opciones	`XREF(SHORT,UNREFS)`
2	External Symbol Dictionary	secciones y nombres externos	`ZTKHOLA SD … 000002A9`
3–5	el fuente con los bytes	clases 1 a 4	170 sentencias
6	Relocation Dictionary	direcciones absolutas a corregir al cargar	**2**: `000025` y `0000B5`
7	Symbol and Literal Cross Reference	dónde se define y usa cada nombre	`RCFINAL … 20M 45M 84M 101`
8	Macro and Copy Code Source Summary	de dónde salió cada macro	8 de `SYS1.MACLIB`
9	Using Map	hasta dónde llegó cada base	R12 hasta `29E`
10	Register Cross Reference	quién usa y cambia cada registro	R4–R11 sólo en 12 y 105
11	Diagnostic … Summary	errores y resumen	`No Statements Flagged` · RC 000

📍 **La RLD**

Las dos `AL3(PRTDCB)` del `OPEN` y el `CLOSE`: z/OS les **suma** la dirección de carga. Base + desplazamiento no necesita nada.

🔤 **Letras de la XREF**

`M` modifica · `B` salto · `U` using · `X` execute · sin letra: lectura.

🔎 **Binder: 681 → 688**

`ALIGN = DBLWORD` y 688 es el primer múltiplo de 8: la explicación más probable del relleno (inferencia, sin mensaje que lo diga).

Fuente: listado de JOB24962 (páginas 1–11) · LKED/SYSPRINT de JOB24962 (Module Map, Save Module Attributes)

Lámina L01-29_el_listado_entero

6. Lo que te llevás de la L01

Con las cinco clases, cualquier renglón de ZTKHOLA tiene respuesta:

Si mirás…	Te preguntás…	Clase
las columnas, el `*`, el `+`	qué clase de sentencia es y en qué parte del programa está	1
`5820 C0CC`	qué instrucción es, qué registro, qué base, qué desplazamiento, cuántos bytes	2
`STM`, `LM`, `BR 14`, R15	cómo entra, cómo sale, qué RC deja	3
`DS`, `DC`, `=F'8'`, `EX`	dónde cae cada dato, cuánto ocupa, cómo llega el PARM	4
`SVC`, el DCB, `AMODE`, la RLD	qué le pide a z/OS, dónde vive, qué corrige el cargador	5

ZTKMON v0.9 tiene **las mismas cinco partes**, el **mismo prólogo y epílogo byte por byte**, las mismas macros de E/S y el mismo `AMODE 31` / `RMODE 24`. Lo que agrega son **tres bases**, **rutinas internas** con `BAL R14`, **memoria pedida**

con `STORAGE OBTAIN,LOC=31` y la lectura de bloques de control de z/OS (L15). Ya tenés con qué leerlo.

Resumen

Idea	En una línea	Testigo
Macro	un pedido armado por IBM; el nivel (`01-`, `02-`) dice quién la trajo	sentencias 30 y 79
SVC	la ventanilla de z/OS: `0A` + número	`OPEN` 19, `WTO` 35, `CLOSE` 20
`PUT`	sin SVC: `BASR` a la rutina que el `OPEN` dejó en DCB+49	sentencias 81–83
DCB	96 bytes; tus parámetros caen en campos fijos; `OPEN` prende `X'10'` en +48	sentencias 125–163
AMODE / RMODE	31 = cómo direcciona; 24 = vive debajo de 16 MB, por las direcciones de 3 bytes del DCB	renglones 2–3 y binder
RLD	sólo 2 direcciones absolutas para corregir al cargar: los `AL3(PRTDCB)`	página 6
Cross Reference	`M` modifica, `B` salto, `U` using, `X` execute	página 7
Binder	`ALIGN = DBLWORD`, `MODULE SIZE 2B0`, RC 0	LKED de JOB24962

Preguntas de comprobación

1. La RLD tiene sólo 2 renglones, pero el programa usa `SALVAR1`, `RCFINAL` y `SALIDA` decenas de veces. ¿Por qué ninguna de esas instrucciones necesita que z/OS la corrija al cargar?
2. Si cambiaras el renglón 3 a `RMODE ANY` y nada más, ¿qué parte del programa dejaría de cumplir lo que pide QSAM? ¿Qué tendrías que cambiar para poder subir el programa arriba de la línea?
3. En la Cross Reference, `SALIDA` dice `67M 68M 80 121M`. ¿Qué hace cada una de esas sentencias con `SALIDA`? ¿Por qué la 80 no lleva `M`, si `PUT` escribe justamente `SALIDA`?

Para corregirte (mirá recién después de contestar)

1. Porque son **base + desplazamiento**: la instrucción guarda «R12 + tanto», no una dirección. La corrección la hace el `LR 12,15` del prólogo, una sola vez, en ejecución.
2. El DCB (adentro de la CSECT) podría quedar arriba de 16 MB, y sus direcciones de 3 bytes no lo podrían alcanzar. Habría que sacar el DCB a memoria pedida con `STORAGE OBTAIN,LOC=24` (o `GETMAIN` debajo de la línea) y apuntarlo con un registro. Qué haría exactamente z/OS con la versión `RMODE ANY` sin cambios **no está medido**.
3. 67 (`MVC SALIDA,ESPACIOS`) y 68 (`MVC SALIDA(11),…`) escriben en `SALIDA`; 121 (`COPIAPRM`) también escribe (es el destino del `MVC`). La 80 es `LA 0,SALIDA`: sólo toma la **dirección**, no toca el dato. `PUT` **lee** `SALIDA` para copiarla al buffer: no la modifica.

Mis respuestas

Relacionado: *L01.4 - Clase 4 - Los datos - DS, DC, literales, el PARM y el EX* · *L01.1 - Clase 1 - La forma de un programa - renglones, columnas y sentencias* ·

Notas